



TestSplit optimises CI pipelines by analysing historical test runtimes and distributing tests across parallel jobs using LPT and MULTIFIT scheduling, achieving a 2.31× speed-up (57% reduction in pipeline time)*.

Students: Samson Oloruntola, Computer Science & Marjia Siddik, Computer Science

Supervisor: Prof Paul M. Clarke

1

THE PROBLEM

CI pipelines run tests one by one.

The bigger your project, the longer you wait.

Total Pipeline Time



* Evaluated on the apache/commons-lang GitHub repository

2

HOW IT WORKS



Parse Test Reports

Reads JUnit XML output from your Maven Build



Smart Test Distribution

Group tests into equal-time parallel jobs to minimise total pipeline time.



Generate CI Config

Outputs a ready-to-use GitHub Actions or GitLab CI YAML File

3

PERFORMANCE IMPACT

TOTAL TESTS	↑ 3236150.0%	SEQ. DURATION	↑ 128296.7%
65904		361.2s	
across 42 profiling runs		unparallelised total	

Before (Sequential Execution)

↓ 57% reduction in pipeline time (2.31× speed-up)

MAKESPAN	↑ 78033.4%	SPEED-UP	
156.27s		2.31×	
critical path - 8 jobs		13147 unstable - 26 outliers	

After (Parallel Execution with TestSplit)

```
jobs:
  build:
    run: mvn clean install -DskipTests

  test-job-1:
    run: mvn test -Dtest=ExtendedMessageFormatTest,FastDateParserTest

  test-job-2:
    run: mvn test -Dtest=StringUtilsTest,ArrayUtilsTest
```

Example generated job split

4

RESULTS

65,904 tests profiled (apache/commons-lang)

57% reduction in CI pipeline time

2.31× speed-up via parallel execution

Validated on real-world open-source CI pipelines

No modification to existing test suites required

